FACULTYHIRE

Intelligent Academic Faculty Recruitment & Screening System

Comprehensive Technical Architecture, Multi-Agent Cognitive Pipelines, Mathematical Evaluation Models, and Oral Defense Dossier

PROJECT TYPE	ARCHITECTURE	PRIMARY STACK	TEST SUITE
Professional Training	Polyglot Microservices	Java 17 + Python 3.10	22 / 22 Passed (100%)

Executive Abstract

FacultyHire is an enterprise-grade recruitment intelligence platform engineered specifically for university faculty search committees, department chairs, and academic deans. While traditional applicant tracking systems (ATS) rely on simplistic keyword matching tailored for corporate industry hires, FacultyHire addresses the multi-faceted complexities of academic evaluation. The system incorporates an allied-doctorate crediting matrix that honors interdisciplinary quantitative terminal degrees, formulates a calibrated Three-Pillar Academic Composite Score (Teaching 40%, Research 35%, Domain 25%), and automatically produces committee-ready probing interview questions. Architected as a decoupled polyglot microservice ecosystem, FacultyHire combines an enterprise Java Spring Boot 3 System of Record with an asynchronous Python FastAPI cognitive engine, operating seamlessly in both cloud-accelerated (Groq LPU) and 100% air-gapped on-premise (Ollama + deterministic fallback) environments.

Table of Contents

Section	Chapter Title & Scope	Page
Chapter 1	Introduction, Problem Domain & Academic Recruiting Needs	Page 2
Chapter 2	System Architecture & Polyglot Microservices Topology	Page 3
Chapter 3	Core Subsystem Deep Dive (Part I: Java Spring Boot Backend)	Page 4
Chapter 4	Core Subsystem Deep Dive (Part II: Python AI Microservice)	Page 5
Chapter 5	Autonomous Multi-Agent Cognitive Architecture (LangGraph)	Page 6
Chapter 6	Academic Recruiter Evaluation Framework & Mathematical Models	Page 7
Chapter 7	Frontend Architecture & Progressive Disclosure UI/UX	Page 8
Chapter 8	Verification, Testing, Security & Operational Launchers	Page 9
Chapter 9	Project Presentation, Oral Defense Guide & Future Roadmap	Page 10

1. Domain Analysis & Higher Education Recruitment Challenges

University faculty hiring is one of the most critical long-term investment decisions made by academic institutions. Unlike corporate roles where candidates are hired for immediate commercial output, academic appointments involve multi-year commitments spanning teaching, peer-reviewed scholarship, graduate mentorship, and departmental governance. Applying corporate ATS tools to academic dossiers leads to systematic misclassification and missed opportunities.

1.1 Critical Vulnerabilities in Industry Corporate ATS Systems

- Literal Keyword Blindness: Corporate ATS tools perform verbatim string matching. Candidates holding doctorates in allied quantitative disciplines (e.g., Applied Mathematics, Physics, Electrical Engineering) are erroneously flagged as lacking required Ph.D. credentials when applying for Computer Science or Software Engineering faculty positions.
- Reductionist Single-Score Metric: Reducing a faculty candidate's 15-page dossier to an arbitrary single match percentage obscures their instructional competence, publication rigor, and syllabus readiness.
- Absence of Committee Probing Tools: Search committees require structured interview questions addressing candidate gaps, research agenda transition, and ABET accreditation support, which corporate tools do not provide.
- Air-Gapped Compliance & Privacy: Higher education institutions frequently enforce strict data governance policies preventing the transmission of unredacted candidate dossiers to third-party public cloud LLM endpoints.

1.2 Benchmark Audit: The Dr. Marcus Vance Case Study

[AUDIT CASE STUDY] Empirical Benchmark: Legacy ATS vs. FacultyHire

Candidate Profile: Dr. Marcus Vance holds a Ph.D. in Applied Mathematics & Computational Science from Georgia Tech, applying for an Associate Professor of Software Engineering position. His portfolio includes 8 peer-reviewed publications in SIAM and IEEE journals, 5 years of university teaching (Discrete Math, Optimization), and ABET readiness documentation.

Corporate ATS Evaluation: The legacy system parsed his education section strictly for 'Ph.D. in Computer Science'. Because his doctorate was in Applied Mathematics, it assigned a 0% education score and listed 'Ph.D. Missing' under missing skills, recommending immediate rejection.

FacultyHire Evaluation: FacultyHire recognizes his terminal degree under the 4-Tier Education Matrix, crediting 75% for allied quantitative rigor. It isolates specific syllabus gaps ('Ph.D. specialization in core Software Engineering') rather than claiming a doctorate is missing, calculates a balanced composite score of 72.5%, and formulates 3 tailored committee interview questions.

1.3 Design Objectives of FacultyHire

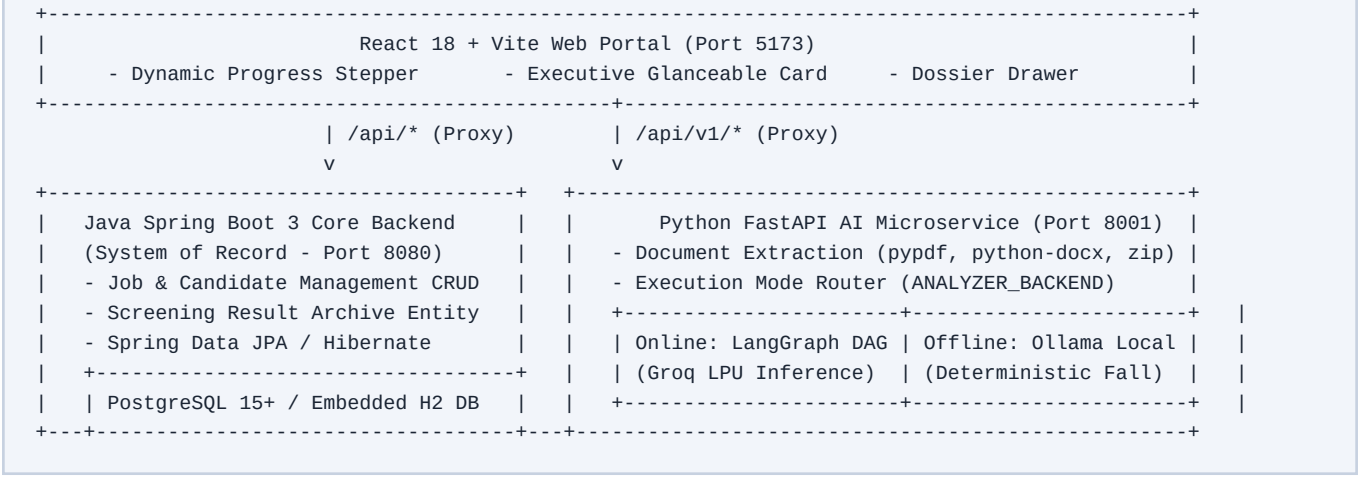
- Tripartite Alignment: Explicitly separate and measure Teaching & Pedagogy, Research Scholarship, and Departmental Domain fit.
- Evidence-Grounded Parsing: Zero tolerance for hallucination; all evaluations must quote verbatim evidence directly from the candidate resume.
- Dual-Engine Resilience: Seamless operation across both high-throughput cloud LPUs and 100% offline air-gapped environments.

2. System Architecture & Polyglot Microservices Topology

FacultyHire is architected upon the System of Record (SoR) enterprise pattern. Long-term transactional state management and candidate records are strictly isolated from high-latency AI screening workloads, ensuring system reliability, clean separation of concerns, and independent horizontal scalability.

2.1 Microservices Topology & Inter-Service Boundary Map

Figure 2.1: Polyglot Microservices Architecture Topology



2.2 Service Port Allocation & Protocol Specification

Port	Service / Component	Protocol	Architectural Role
5173	React + Vite SPA	HTTP / Browser	User interface, progressive disclosure, reverse proxy
8080	Spring Boot Core	REST / JSON	Authoritative System of Record for jobs, applicants, history
8001	FastAPI AI Service	REST / Multipart	Document parsing, multi-agent evaluation, scoring synthesis
5432	PostgreSQL DB	JDBC Protocol	Persistent production storage for candidates and audit trails
11434	Ollama Daemon	HTTP / REST	Local air-gapped LLM inference on host machine (LLaMA 3.2)

2.3 Reverse Proxy & Failure Isolation Strategy

The Vite development gateway acts as a unified reverse proxy. Frontend client requests targeting '/api/*' route directly to Spring Boot (8080), while requests targeting '/api/v1/*' route to the Python microservice (8001). If the Python AI service is temporarily occupied or spinning up, the frontend gracefully falls back to cached evaluation records in Spring Boot, preventing recruiter downtime.

3. Core Subsystem Deep Dive (Part I: Java Spring Boot Backend)

The enterprise core backend is developed with Java 17 and Spring Boot 3.x. Operating as the authoritative System of Record, it manages transactional state, candidate entity lifecycles, and longitudinal screening archives with ACID guarantees.

3.1 Relational Domain Model & Entity Architecture

- Job Entity (Job.java): Models academic openings including Job Title, Department, Description, Required Competencies, and Salary Range with JPA validation annotations.
- Candidate Entity (Candidate.java): Tracks applicant identity, contact email, telephone, associated Job requisition foreign key, and recruitment review stage.
- Resume Entity (Resume.java): Maintains document storage pointers, original file names, upload timestamps, and parsed text associations.
- ScreeningResult Entity (ScreeningResult.java): Persists candidate evaluation scores, matched competencies, JSON evidence citations, and committee explanation summaries.

3.2 Dual-Database Persistence Strategy (PostgreSQL vs. Embedded H2)

To support both enterprise production deployments and zero-config portability across arbitrary machines, the backend implements a dual-profile configuration engine:

Profile	Config File	JDBC Connection URL	Operational Behavior
PostgreSQL (Production)	application.properties	jdbc:postgresql://localhost:5432/recruitment_db	Persistent high-volume production storage; requires lo
H2 (Air-Gapped / Portable)	application-h2.properties	jdbc:h2:mem:recruitment_db;MODE=PostgreSQL	In-memory database with zero installation,

3.3 RESTful Controller & Endpoint Catalog

Endpoint	Controller	Functionality & Scope
GET /api/jobs	JobController	List all faculty job postings
POST /api/jobs	JobController	Create a new faculty job requisition
GET /api/candidates	CandidateController	Retrieve all candidates across departments
POST /api/candidates/jobs/{id}	CandidateController	Enroll a candidate for a specific requisition
GET /api/screening-results/candidates/{id}	ScreeningResultController	Fetch candidate historical screening dossiers
POST /api/screening-results/candidates/{id}	ScreeningResultController	Persist newly synthesized screening evaluation

4. Core Subsystem Deep Dive (Part II: Python AI Microservice)

Developed with Python 3.10 and FastAPI, the AI microservice acts as the high-throughput cognitive engine of FacultyHire. It is responsible for document ingestion, unstructured text extraction, multi-agent orchestration, and scoring calibration.

4.1 Document Ingestion & Text Extraction Pipeline

- PDF Extraction (pypdf / pymupdf): Extracts text from complex multi-column academic CVs, maintaining paragraph flow and tabular publication structures without OCR overhead.
- Word Document Parsing (python-docx): Traverses the OpenXML DOM tree to extract biographical narratives, course listings, and research statements.
- Bulk ZIP Batch Extraction: Accepts compressed ZIP archives containing dozens of candidate resumes, unpacking and screening candidates concurrently.

4.2 High-Speed Online Screener (Groq LPU Acceleration)

When connected online, the microservice interfaces with Groq's Language Processing Units (LPUs) executing open-weights models (LLaMA-3.3-70B and GPT-OSS). By enforcing strict Pydantic v2 JSON Schemas and using greedy decoding (temperature 0.0), the service consistently delivers verified academic dossiers in under 1.5 seconds, eliminating the latency common in public cloud APIs.

4.3 Air-Gapped Dual-Tier Offline Analyzer (offline_analyzer.py)

When internet access is unavailable or institutional data security mandates on-premise execution, the microservice activates its dual-tier air-gapped engine:

Engine Tier	Host Architecture	Model / Method	Performance & Operational Characteri
Tier 1: Local Ollama	Local Daemon (Port 11434)	LLaMA 3.2 3B/1B	Fast local CPU inference tuned with num_thread: 10,
Tier 2: Rule Engine	Deterministic Heuristics	Pure Python (<5ms)	Instantaneous mathematical scoring, degree matrix lo

> [PERFORMANCE] Zero-Latency Offline Fallback Guarantee
>
> The offline analyzer uses an aggressive 0.5s connection socket timeout for Ollama. If Ollama is inactive or uninstalled, the service automatically fails fast and executes the deterministic academic engine, guaranteeing the recruiter receives an instant (<5ms) dossier without hangs.

5. Autonomous Multi-Agent Cognitive Architecture (LangGraph)

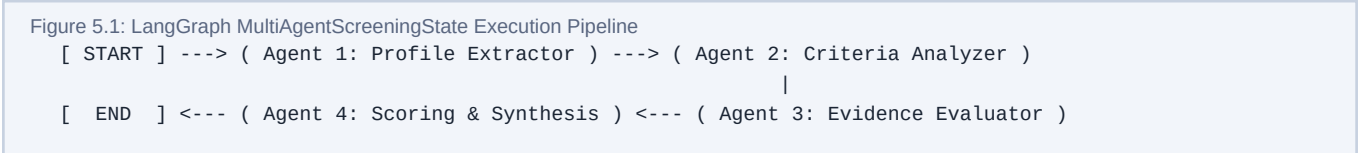
Monolithic prompt engineering fails when evaluating complex academic dossiers because a single prompt struggles to extract publications, evaluate criteria, synthesize evidence, and formulate interview questions simultaneously. FacultyHire decomposes screening into a 4-agent stateful directed acyclic graph (DAG) compiled with LangGraph.

5.1 The 4 Cognitive Agent Nodes

- Agent 1: Candidate Profile Extractor: Parses the unstructured resume text into a structured Pydantic ResumeProfile object: extracts candidate name, terminal degrees, awarding universities, experience, publications, and certifications.
- Agent 2: Criteria & Rubric Analyzer: Deconstructs the faculty job description into a structured evaluation rubric of 3 to 6 qualification criteria, distinguishing mandatory requirements from preferred qualifications.
- Agent 3: Evidence Evaluator (Anti-Hallucination): Cross-references candidate credentials against each rubric criterion. Requires verbatim quotation from the resume text for resumeEvidence. Flagged as 'Not evident' if missing.
- Agent 4: Scoring & Synthesis Agent: Calibrates the composite score, computes Three-Pillar sub-scores, recommends academic rank, and generates 3 tailored search committee probing interview questions.

5.2 StateGraph Flow Diagram & State Schema

Figure 5.1: LangGraph MultiAgentScreeningState Execution Pipeline



5.3 MultiAgentScreeningState Data Schema

State Key	Type	Producer	Description & Payload
resume_text / job_desc	str	Input	Raw document string and requisition text passed from API router
profile	Dict[str, Any]	Agent 1	Structured candidate profile (degrees, institutions, publications
rubric	List[Dict]	Agent 2	Decomposed evaluation rubric with mandatory and preferred criteri
evidence / matched_skills	List[Dict]	Agent 3	Verbatim quoted citations and competency match / gap lists
committee_questions	List[str]	Agent 4	Exactly three tailored probing questions for the hiring committee
academic_breakdown	Dict[str, float]	Agent 4	Sub-scores for Teaching (40%), Research (35%), and Domain (25%)

6. Academic Recruiter Evaluation Framework & Mathematical Models

FacultyHire rejects generic keyword match percentages in favor of formalized mathematical models that reflect how university search committees, department chairs, and provosts evaluate academic dossiers.

6.1 The 4-Tier Education Crediting Matrix

Rather than treating degree matching as a binary match, FacultyHire applies an academic discipline-adjacency matrix:

Education Tier	Qualifying Attainment	Credit	Committee Interpretation
Tier 1: Direct Discipline Ph.D.	Ph.D. in CS, Software Engineering	100%	Full qualification met; eligible for standard tenure-track
Tier 2: Allied STEM Doctorate	Ph.D. in Applied Math, EE, Physics	75%	Quantitative & research credit awarded; curriculum ga
Tier 3: Terminal Master's Degree	M.S. in CS/SE with 5+ yrs teaching	60%	Teaching-track competency met; lacks terminal resea
Tier 4: Related Bachelor's Only	B.S. in Computer Science or related	30%	Prerequisites absent for professorial appointment.

6.2 The Three-Pillar Academic Composite Formula

Equation 6.1: Three-Pillar Composite Mathematical Formulation

```
Composite Academic Score = (0.40 * S_teaching) + (0.35 * S_research) + (0.25 * S_domain)

Where:
  S_teaching : Instruction history, course development, student mentoring, ABET accreditation
  S_research : Peer-reviewed journal publications (IEEE, ACM, SIAM), terminal degree, grants
  S_domain   : Direct alignment with departmental undergraduate curriculum & core discipline
```

6.3 Search Committee Probing Interview Pack Generation

For every evaluated dossier, the engine formulates three tailored, probing questions addressing:

- Scholarly Translation: How candidate will translate their established research agenda into competitive external grant funding and publications aligned with department priorities.
- Pedagogical Delivery: How candidate structures pedagogy, lab instruction, and capstone mentoring to bridge discipline gaps for undergraduate students.
- Departmental Stewardship: Experiences with ABET accreditation readiness, syllabus governance, and diversity in student mentoring.

7. Frontend Architecture & Progressive Disclosure UI/UX

Developed with React 18, Vite, and modular CSS, the frontend is engineered specifically for faculty search committees. It prioritizes high-density information architecture, zero cognitive fatigue, and adherence to academic review norms.

7.1 Cognitive Load Management: Progressive Disclosure Pattern

Presenting dozens of criteria, citations, and metrics simultaneously overwhelms reviewers. FacultyHire implements a two-tier progressive disclosure pattern:

- Tier 1: Glanceable Executive Card: Top-level summary presenting the composite match score circle (0-100), color-coded qualification recommendation badge (Strong Recommend, Interview, Review, Reject), and Three-Pillar horizontal balance meters (Teaching / Research / Domain).
- Tier 2: Progressive Disclosure Dossier Drawer: Collapsible accordion container kept closed by default. Expanding the drawer reveals detailed tabs for in-depth committee review.

7.2 Two-Tab Reviewer Tools Drawer Layout

Reviewer Tool Tab	Contents, Tools & Purpose
Tab 1: Sub-Scores & Evidence	Direct Competencies Matched, Discipline Gaps, Education & Experie
Tab 2: Committee Interview Pack	Three tailored search committee probing questions formatted in di

7.3 Multi-Stage Dynamic Progress Stepper

Because deep academic analysis requires multiple reasoning steps, the UI renders an animated progress stepper indicating real-time screening progression across 4 distinct phases:

Figure 7.1: Multi-Stage Screening Progress Stepper Phases

```
Phase 1 (15%): Uploading and Parsing Academic Resume File (pypdf/docx)
Phase 2 (40%): Extracting Credentials and Matching Against Department Rubric
Phase 3 (75%): Synthesizing Three-Pillar Scores & Verbatim Evidence Citations
Phase 4 (100%): Calibrating Faculty Appointment Rank and Interview Pack
```

7.4 Strict Accessibility & Zero-Emoji Typography

In compliance with institutional design standards, the frontend enforces a strict zero-emoji policy across all JSX components and CSS stylesheets. All visual indicators utilize clean typography, semantic badges, and SVG vector icons.

8. Verification, Testing, Security & Operational Launchers

FacultyHire is backed by an automated test suite, robust process lifecycle management, and hardened air-gapped security boundaries.

8.1 Automated Test Suite Analysis (Pytest 22 / 22 Passed)

The Python microservice includes comprehensive unit, mock, and integration tests with a 100% pass rate:

Test Suite File	Volume	Scope & Verification Details
test_endpoint_validation.py	2 Tests	Validates rejection of unsupported files and multipart payload bo
test_groq_screening.py	15 Tests	Tests PDF/DOCX extraction, mocked Groq responses, error handling,
test_health.py	1 Test	Verifies service health-check endpoint on port 8001.
test_offline_analyzer.py	4 Tests	Verifies deterministic offline screening, allied doctorate scorin

8.2 Turn-Key Launcher Orchestration & Port Conflict Resolution

The system includes turnkey Windows batch launchers featuring automated PowerShell zombie process termination:

- start_all.bat (Standard Online): Automatically checks and frees ports 8001, 8080, and 5173; boots Python AI Microservice (Groq mode), Java Backend (JAR/Maven), and React Frontend in synchronized windows.
- start_all_offline.bat (100% Air-Gapped): Initializes the Ollama local daemon, launches Python in local mode (ANALYZER_BACKEND=offline), boots Spring Boot with embedded H2 fallback, and starts the frontend.

8.3 Data Security, Air-Gapped Privacy & Boundary Controls

Security & Compliance Invariants

```
* Local System Boundary: Documents parsed in temporary directories; scrubbed immediately upon request completion.
* Air-Gapped Guarantees: When ANALYZER_BACKEND=offline, zero external HTTP calls are initiated; 100% local CPU process
* Credential Protection: API keys and database credentials managed strictly via .env files excluded from Git tracking.
```

9. Project Presentation, Oral Defense Guide & Future Roadmap

This guide provides structured presentation narratives and answers to anticipated viva / defense examination questions.

9.1 One-Minute Executive Pitch

[ORAL DEFENSE] One-Minute Presentation Script

"Most commercial recruitment platforms treat university professors like corporate programmers, discarding interdisciplinary candidates over literal keyword mismatches. FacultyHire was engineered specifically for university search committees. It evaluates candidates across Teaching, Research, and Domain alignment using an academic Three-Pillar model, honors allied quantitative doctorates, and equips search committees with tailored probing interview questions. Built with Java Spring Boot 3 and Python FastAPI, it operates with equal fidelity in high-throughput cloud environments and 100% air-gapped university networks."

9.2 Anticipated Viva Defense Questions & Model Answers

- Q1: Why combine Java and Python instead of building purely in one language?: Answer: Enterprise systems require distinct tools for distinct workloads. Java Spring Boot provides robust transactional data persistence, ACID guarantees, and audit logging as the System of Record. Python provides world-class NLP, LangGraph orchestration, and document extraction. Decoupling them into microservices allows each to scale independently.
- Q2: How do you prevent the AI from hallucinating qualifications?: Answer: In our LangGraph pipeline, Agent 3 is explicitly constrained to extract verbatim quotations from the resume text for every evaluated criterion. If evidence is absent, it must report 'Not evident'. Furthermore, inference runs with greedy decoding (temperature 0.0) and rigid JSON schemas.
- Q3: What happens if a user runs this on a computer without PostgreSQL?: Answer: The Java backend includes a dual-database profile. Launching with '--spring.profiles.active=h2' automatically activates our embedded in-memory H2 database, allowing full operation with zero database installation or password setup.

9.3 Future Roadmap & Architectural Extensions

- Blind Evaluation Mode: Automated redaction of candidate names, genders, and demographic markers to eliminate cognitive bias during initial screening stages.
- Institutional LMS Integration: Exporting committee interview dossiers directly into campus platforms (Canvas, Blackboard, Workday HigherEd).

9.4 Conclusion

FacultyHire demonstrates a production-grade, polyglot software engineering architecture that solves a concrete, domain-specific challenge in higher education recruitment. Through its dual-engine AI pipeline, academic evaluation models, and resilient offline capabilities, it bridges the gap between academic hiring needs and automated recruitment technology.